

VisionInspect AI: Manufacturing Defect Detection & Quality Inspection System

Infosys Springboard Internship 7.0

Milestone 1

Team 1

Leela Sowmya Pappu

Table of Contents

S.No	Title	Page No
1	Executive Summary	1
2	File Strcuture	2
3	Image Processing Module	2
4	Image Quality Analysis	3
5	Feature Extraction Module	4
6	Inspection Pipeline	5
7	Outcomes & References	6

1. Summary & Scope Differentiation

While **Milestone 1** focused on initial environment setup, basic dataset loading, and basic Exploratory Data Analysis (EDA), **Milestone 2** transitions the system into an end-to-end computer vision feature and quality inspection pipeline.

Capabilities	Milestone 1 (Completed)	Milestone 2
Dataset Handling	Basic path verification & image counts	Advanced dataset summary & defect distribution analytics
Preprocessing	Simple resize (256x256) & basic [0,1] normalization	Multi-stage pipeline: CLAHE, Non-Local Means Denoising, and Normalization
Image Assessment	Visual shape and data type verification	Automated quality scoring (Blur variance, Noise estimation, Contrast, Brightness)
Feature Extraction	None	Multi-modal features (Color histograms, LBP textures, Edge density, Otsu contours)
Workflow Engine	Manual script execution	Unified pipeline.py emitting structured JSON and CSV inspection artifacts

2. File Structure

The visualization and image processing directory was structured to maintain a strict separation of concerns, providing a clean contract for backend and model training engineers:

```
backend/visualization/src/  
├─ config.py           # System constants, image sizes, and output  
directory paths  
├─ preprocessing.py    # Core image enhancement (CLAHE, Denoising,  
Normalization)  
├─ image_quality.py     # Automated image quality evaluation (Blur,  
Noise, Brightness)  
├─ feature_extraction.py # Traditional CV feature extractors (Color,  
LBP, Canny, Otsu)  
├─ statistics.py        # Category and defect distribution analysis  
└─ pipeline.py          # Unified execution workflow generating reports  
and visualizations
```

3. Advanced Image Preprocessing Module (preprocessing.py)

To ensure robust performance under non-ideal factory floor lighting, the basic resizing step from Milestone 1 was expanded into a dynamic enhancement pipeline:

- **Contrast Limited Adaptive Histogram Equalization (CLAHE):** Enhances local contrast in L-space (LAB color mode) to expose faint surface cracks.
- **Fast Non-Local Means Denoising:** Removes camera sensor noise while preserving structural edges.
- **Standardized Resizing & Range Scaling:** Maps images to 256x256 resolution and scales pixel intensities to [0, 1].

Code Snippet:

```
import cv2  
import numpy as np  
from src.config import IMAGE_SIZE  
  
def enhance_contrast(image, clip_limit=2.0, tile_grid_size=(8, 8)):  
    """Apply CLAHE in LAB space to enhance subtle surface defects."""  
    lab = cv2.cvtColor(image, cv2.COLOR_RGB2LAB)  
    l, a, b = cv2.split(lab)  
  
    clahe = cv2.createCLAHE(clipLimit=clip_limit, tileGridSize=tile_grid_size)  
    l = clahe.apply(l)  
  
    enhanced = cv2.merge([l, a, b])
```

```

    return cv2.cvtColor(enhanced, cv2.COLOR_LAB2RGB)

def preprocess_image(image_path, use_denoising=False, use_clahe=True):
    """Main preprocessing pipeline for downstream inference and features."""
    image = load_image(image_path)
    image = resize_image(image, size=IMAGE_SIZE)

    if use_denoising:
        image = cv2.fastNLMMeansDenoisingColored(image, None, 5, 5, 7, 21)
    if use_clahe:
        image = enhance_contrast(image)

    return image.astype(np.float32) / 255.0

```

4. Automated Image Quality Analysis Module (image_quality.py)

In production environments, degraded input images lead to false defect predictions. This module acts as an automated quality gate:

- **Blur Score:** Computed using the Variance of Laplacian. Lower scores indicate blurry camera capture.
- **Noise Estimation:** Standard deviation of differences between the original grayscale image and a Gaussian-blurred variant.
- **Brightness & Contrast:** Mean pixel intensity and intensity standard deviation.

Code Snippet:

```

import cv2
import numpy as np

def calculate_blur_score(image):
    """Estimate image sharpness using Variance of Laplacian. Higher = Sharper."""
    gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
    return float(cv2.Laplacian(gray, cv2.CV_64F).var())

def estimate_noise(image):
    """Estimate image noise level via residual standard deviation."""
    gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
    blur = cv2.GaussianBlur(gray, (3, 3), 0)
    noise = gray.astype(np.float32) - blur.astype(np.float32)
    return float(np.std(noise))

def generate_quality_report(image):
    """Generate complete image quality metrics dict."""
    return {
        "brightness": float(np.mean(cv2.cvtColor(image, cv2.COLOR_RGB2GRAY))),
        "contrast": float(np.std(cv2.cvtColor(image, cv2.COLOR_RGB2GRAY))),
        "blur_score": calculate_blur_score(image),
    }

```

```

        "noise_level": estimate_noise(image)
    }

```

5. Classical & Statistical Feature Extraction Module (feature_extraction.py)

This engine extracts baseline computer vision representations prior to deep learning inference:

- **Color Distribution:** Normalized RGB channel histograms (32 bins per channel).
- **Texture Features:** Local Binary Patterns (LBP) with uniform patterns to detect surface roughness variations.
- **Edge Density:** Canny edge detection density ratio to quantify cracks and fractures.
- **Contour Shapes:** Otsu thresholding coupled with contour area summation to register structural anomalies.

Code Snippet:

```

import cv2
import numpy as np
from skimage.feature import local_binary_pattern
from src.config import HISTOGRAM_BINS, LBP_RADIUS, LBP_POINTS

def extract_texture_features(image, radius=LBP_RADIUS, points=LBP_POINTS):
    """Extract uniform Local Binary Pattern (LBP) texture representation."""
    gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
    lbp = local_binary_pattern(gray, points, radius, method="uniform")

    histogram, _ = np.histogram(
        lbp.ravel(),
        bins=np.arange(0, points + 3),
        range=(0, points + 2)
    )
    histogram = histogram.astype(np.float32)
    histogram /= (histogram.sum() + 1e-6)
    return histogram

def extract_edge_features(image):
    """Calculate ratio of edge pixels using Canny Edge Detection."""
    gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
    edges = cv2.Canny(gray, 100, 200)
    return float(np.count_nonzero(edges) / edges.size)

```

6. Comprehensive Execution & Inspection Pipeline (pipeline.py)

The pipeline.py entry point orchestrates image loading, quality scoring, feature extraction, data logging, and visualization generation.

Code Snippet:

```
import json
import pandas as pd
from src.config import OUTPUT_DIR
from src.preprocessing import load_image, preprocess_image
from src.image_quality import generate_quality_report
from src.feature_extraction import extract_all_features

def run_pipeline(image_path):
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    original = load_image(image_path)
    processed = preprocess_image(image_path)

    # Run Quality Analysis & Save Artifacts
    quality = generate_quality_report(original)
    with open(OUTPUT_DIR / "quality_report.json", "w") as f:
        json.dump(quality, f, indent=4)

    # Extract Features & Save Dataframe
    features = extract_all_features(original)
    feature_summary = {
        "color_features": len(features["color"]),
        "texture_features": len(features["texture"]),
        "edge_density": features["edge_density"],
        "contours": features["shape"]["contour_count"],
        "contour_area": features["shape"]["total_contour_area"],
    }
    pd.DataFrame([feature_summary]).to_csv(OUTPUT_DIR / "feature_summary.csv", index=False)

    return {
        "quality_report": quality,
        "features": features,
        "output_directory": str(OUTPUT_DIR)
    }
```

7. Output Artifacts & Integration Readiness

The execution of Milestone 2 generates structured visual and tabular artifacts under backend/inspection_plots/:

```
backend/inspection_plots/
├── quality_report.json      # JSON metadata containing blur, noise, and lighting scores
├── feature_summary.csv     # Tabular summary of LBP, color, edge, and contour metrics
├── original.png           # Raw RGB image export
├── preprocessed.png       # CLAHE-enhanced preprocessed image export
├── comparison.png        # Side-by-side comparative analysis plot
├── rgb_histogram.png      # Color balance plot across channels
├── grayscale_histogram.png # Pixel intensity spread plot
└── edge_detection.png     # Canny edge overlay plot
```

Key Milestone 2 Outcomes & Next Steps

Summary of Completed Objectives

- Implemented multi-stage computer vision preprocessing pipelines (CLAHE, Denoising, Normalization).
- Built an automated image quality inspection engine to reject poor-quality input images.
- Created feature extraction routines capturing texture (LBP), edge density (Canny), shape (Contours), and color distributions.
- Delivered a unified execution pipeline producing JSON and CSV audit trails alongside visual inspection plots.

References

1. MVTec AD Dataset
<https://www.mvtec.com/company/research/datasets/mvtec-ad>
2. Python Standard Lib Documentation
<https://docs.python.org/3/>
3. OpenCV Documentation
<https://docs.opencv.org/>
4. NumPy Documentation
<https://numpy.org/doc/>
5. Matplotlib Documentation
<https://matplotlib.org/>
6. VisionInspectAI GitHub Repository
<https://github.com/GKSJ-Deepvision/VisionInspectAI>

